

Reflection Log

Noorinder

Credit 3

Assignment Name: UEmployee, Faculty, University

How has your program changed from planning to coding to now?

At first, the program was planned to simulate a university employee management system using inheritance and polymorphism. The base class `UEmployee` was designed to hold common attributes like name and salary, while subclasses `Faculty` and `Staff` introduced specific fields like department and job title. The `University` class served as the user interface, allowing users to display details of employees through a menu system.

1. Missing Override Annotation

- **Problem:** The `getDetails()` method in subclasses lacked the `@Override` annotation. This caused confusion because it wasn't immediately clear that these methods were overriding an abstract method in `UEmployee`.
- **Fix:** Added the `@Override` annotation to explicitly indicate the inheritance relationship.

Before Fix:

```
public String getDetails() {  
    return super.toString() + ", Department: " + departmentName;  
}
```

After Fix:

```
@Override  
public String getDetails() {  
    return super.toString() + ", Department: " + departmentName;  
}
```

Reflection: This change improved the readability and clarity of the code by explicitly showing which methods were part of the inheritance structure.

2. Incorrect Initialization of Salary

- **Problem:** The `Faculty` constructor didn't pass the salary to the `UEmployee` constructor, resulting in incorrect data for faculty members.
- **Fix:** Updated the `Faculty` constructor to include the salary parameter and passed it to the parent class using `super()`.

Before Fix:

```
public Faculty(String name, String departmentName) { // Missing salary parameter
    super(name, 0); // Salary was set to 0 by default
    this.departmentName = departmentName;
}
```

After Fix:

```
public Faculty(String name, double salary, String departmentName) { // Salary parameter added
    super(name, salary); // Passes salary to UEmployee constructor
    this.departmentName = departmentName;
}
```

Reflection: Properly initializing inherited fields ensures data consistency and eliminates errors in the program's output.

3. Invalid Employee Selection

- **Problem:** If the user entered an invalid employee number, the program either threw a `NullPointerException` or displayed incorrect data. This happened because the program didn't handle invalid inputs in the `switch` statement.
- **Fix:** Added a `default` case in the `switch` statement to handle invalid inputs gracefully.

Before Fix:

```
switch (empNum) {
    case 1:
        employee = faculty;
        break;
    case 2:
        employee = staff;
        break;
    // No handling for invalid inputs
}
```

After Fix:

```
switch (empNum) {  
    case 1:  
        employee = faculty;  
        break;  
    case 2:  
        employee = staff;  
        break;  
    default:  
        System.out.println("Invalid Employee number!");  
        continue; // Re-prompt the user  
}
```

Reflection: This improvement prevents crashes and ensures the program remains user-friendly by handling unexpected inputs.

4. Validation for Negative Salaries

- **Problem:** The `setSalary()` method didn't validate the salary value, allowing negative salaries to be assigned. This led to unrealistic and incorrect data for employees.
- **Fix:** Added a validation check to ensure only non-negative salary values are accepted.

Before Fix:

```
public void setSalary(double salary) {  
    this.salary = salary; // No validation for negative values  
}
```

After Fix:

```
public void setSalary(double salary) {  
    if (salary >= 0) {  
        this.salary = salary; // Assign valid salaries  
    } else {  
        throw new IllegalArgumentException("Salary cannot be negative.");  
    }  
}
```

Reflection: Adding validation ensures data integrity and prevents users from entering invalid or unrealistic values.

5. Lack of a Clear Quit Option

- **Problem:** The menu didn't include an explicit quit option, causing confusion for users about how to exit the program.
- **Fix:** Added a **Q** option to the menu, allowing users to quit the program easily and intuitively.

Before Fix:

```
do {  
    System.out.println("(E)mployee Details");  
    action = input.next();  
    // No clear quit option  
} while (true);
```

After Fix:

```
do {  
    System.out.println("(E)mployee Details | (Q)uit");  
    action = input.next();  
} while (!action.equalsIgnoreCase("Q"));  
System.out.println("Exiting program.");
```

Reflection: This change made the program more user-friendly by providing a clear way to terminate the session.